CSE4088 – Introduction to Machine Learning

2022-2023

Homework 2

Generalization Error

1.

$$P[|E_{in}(g) - E_{out}(g)| > 0.05] \leq 2e^{-2(0.05)^2 N} \leq 0.03$$

$$\Rightarrow e^{-2(0.05)^2 N} \leq 0.015$$

$$\Rightarrow -2(0.05)^2 N \leq \ln(0.015)$$

$$\Rightarrow N \geq \frac{\ln(0.015)}{-0.005}$$

$$\Rightarrow N \geq 839.94 \approx \boxed{1000}$$

2.

$$P[|E_{in}(g) - E_{out}(g)| > 0.05] \leq 2 \cdot 10e^{-2(0.05)^2 N} \leq 0.03$$

$$e^{-2(0.05)^2 N} \leq 0.0015$$

$$-2(0.05)^2 N \leq \ln(0.0015)$$

$$N \geq \frac{\ln(0.0015)}{-2(0.05)^2} \Rightarrow N \geq 1300.45 \approx \underline{1500}$$

3.

$$N \geq \frac{\ln(0.00015)}{-0.005} \Rightarrow N \geq 1760.975 \approx \boxed{2000}$$

Perceptron Learning Algorithm

```python
def experiment(num_steps: int, num_points: int):
    num_iters = []  # store number of iterations for each run
    num_correct = 0  # number of correct classified points
    num_wrong = 0  # number of wrong classified points
    for i in range(num_steps):
        ds = Dataset(num_points=num_points)  # generate dataset with 10 points
        pla = PLA(dataset=ds)  # construct perceptron learning algorithm
        num_iters.append(pla.fit())  # add the current number of iterations to the list
        # iterate over each point to check they are correctly classified or not
        for j in range(num_steps):
            p = random_point()
            # if two results are equal the point is correctly classified
            if target_func(ds.target_a, ds.target_b, p) == pla.candidate_func(p):
                num_correct += 1
            else:
                num_wrong += 1

    return num_iters, num_correct, num_wrong


total_num_iters, total_num_correct, total_num_wrong = experiment(num_steps=1000, num_points=10)

print("Mean number of iterations for each run: ", np.mean(total_num_iters))
rate_wrong = float(total_num_wrong) / (total_num_wrong + total_num_correct)
print("P[f(x)≠g(x)]: ", rate_wrong)
print("P[f(x)=g(x)]: ", 1.0 - rate_wrong)

```

For each run I created a dataset with two features. Each dataset contains 10 random generated points. Perceptron learning algorithm is run for 1000 times. Average number of iterations, correct classifications and wrong classifications are calculated and printed to the screen.

4. N=10

```
Mean number of iterations for each run:  9.325
P[f(x)≠g(x)]:  0.111939
P[f(x)=g(x)]:  0.888061
```

The closest result which is 15 is given in [b].

5. N=10

```
Mean number of iterations for each run:  9.325
P[f(x)≠g(x)]:  0.111939
P[f(x)=g(x)]:  0.888061
```

P[f(x)≠g(x)]: 0.111939

The closest result which is 0.1 is given in [c].

6. N=100

```
Mean number of iterations for each run:  104.913
P[f(x)≠g(x)]:  0.013504
P[f(x)=g(x)]:  0.986496
```

The closest result which is 100 is given in [b].

7. N=100

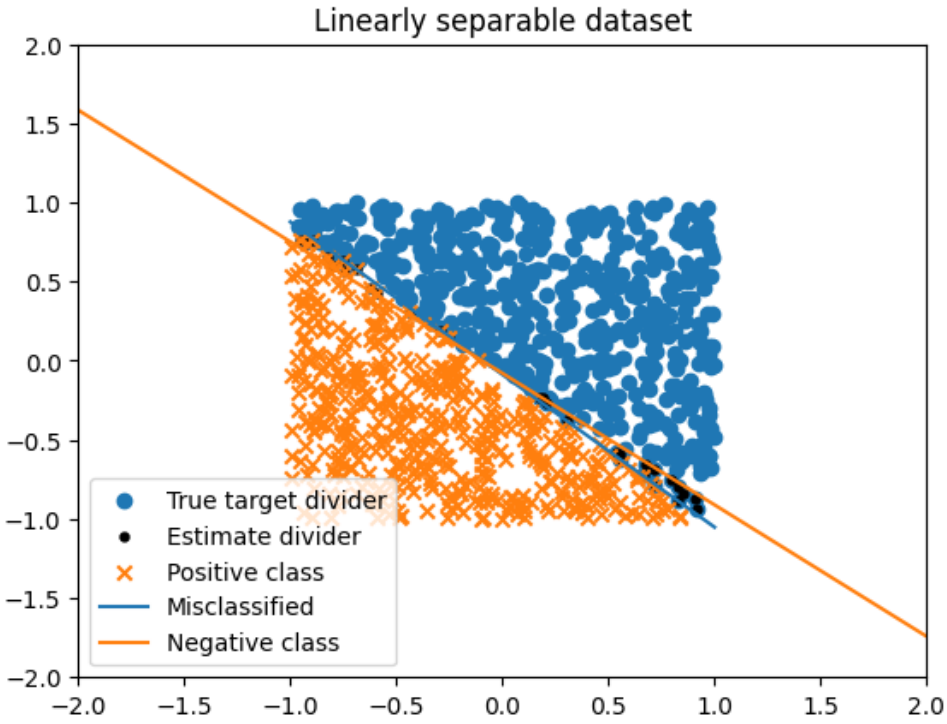
```
Mean number of iterations for each run:  104.913
P[f(x)≠g(x)]:  0.013504
P[f(x)=g(x)]:  0.986496
```

P[f(x)≠g(x)]: 0.013504

The closest result which is 0.01 is given in [b].

Linear Regression

8. Firstly, we must create a dataset and be sure it is linearly separable. Example linearly separable dataset is given below.



Then we run linear regression with two features 1000 times using a different dataset each time. In each run we calculate the inaccuracy of the classifier after the final step we can calculate the average inaccuracy.

Pseudo code is given below.

```python
import numpy as np


class LinearRegression:
    def __init__(self, dataset):
        self.E_in = 0
        self.dataset = dataset
        # Implement linear regression here
        pass


def generate_linearly_separable_data(sample_size: int, min_val: int, max_val: int):
    # Generate uniformly distributed dataset here
    return []


N = 100  # sample size
E_ins = []  # store E_in values

# run the experiment 1000 times
for i in range(1000):
    # Generate uniformly distributed linearly separable dataset for the current run
    dataset = generate_linearly_separable_data(10, -1, 1)
    # run linear regression on the current dataset
    linear_classifier = LinearRegression(dataset=dataset)
    # save the current E_in value to calculate average later
    E_ins.append(linear_classifier.E_in)

# calculate the average of all results
average_E_in = np.mean(E_ins)

print(average_E_in)
```

9. By using the same method in the previous question this time, we must run the same experiment with a bigger sample size which is 1000. We just need to change one line in the code.

```python
N = 1000  # sample size
```

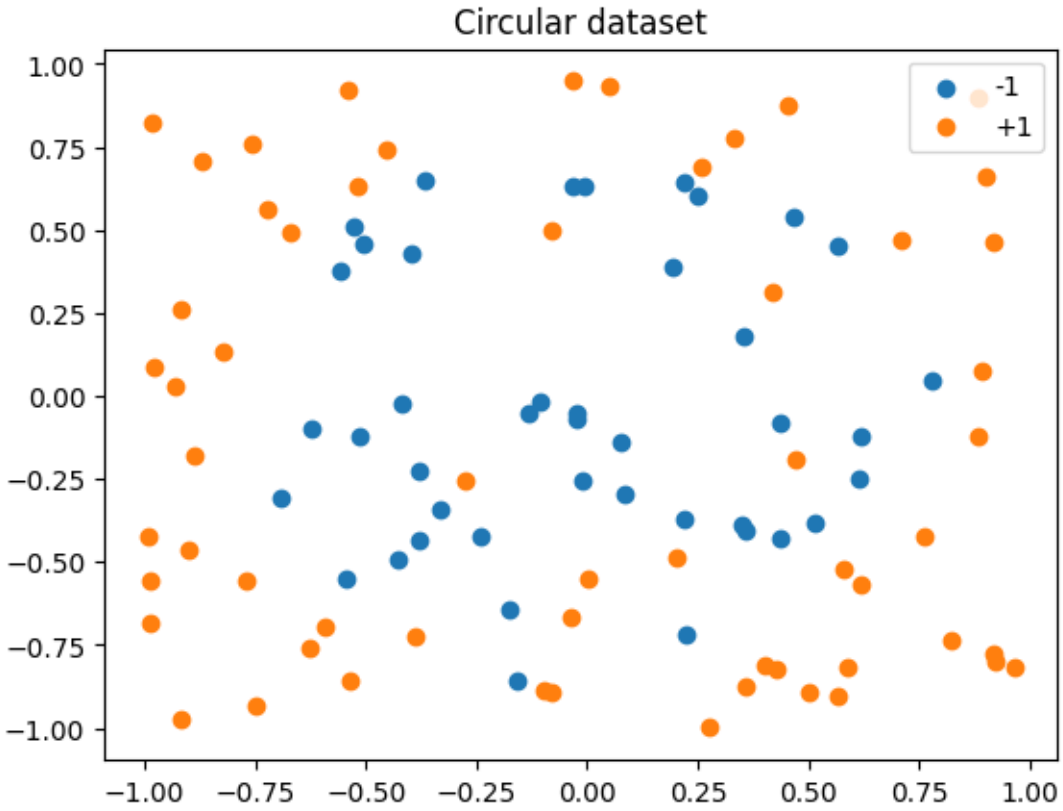
10. -

Nonlinear Transformation

11. This time we must use nonlinear dataset rather than linearly separable dataset we used in the linear regression part.

We can use a circular data for this problem.

For example:



This dataset cannot be classified with a line.

Pseudocode is given below.

```
import numpy as np


class LinearRegression:
    def __init__(self, dataset):
        self.E_in = 0
        self.dataset = dataset
        pass

    def fit(self):
        # Implement linear regression here
        pass


def generate_circular_dataset(sample_size: int, min_val: int, max_val: int):
    # Generate uniformly distributed dataset here
    return []


N = 1000  # sample size
E_ins = []  # store E_in values

# run the experiment 1000 times
for i in range(1000):
    # Generate uniformly distributed linearly separable dataset for the current run
    dataset = generate_circular_dataset(N, -1, 1)
    # run linear regression on the current dataset
    linear_classifier = LinearRegression(dataset=dataset)
    linear_classifier.fit()
    # save the current E_in value to calculate average later
    E_ins.append(linear_classifier.E_in)

# calculate the average of all results
average_E_in = np.mean(E_ins)

print(average_E_in)
```

12. -

13. -